Step-by-Step Walkthrough

Step 1 — Load & Pre-process the Data

The CSV has 48 monthly rows (Oct 2020 – Sep 2024). After parsing the date strings into proper Timestamp objects and sorting chronologically, we create a **numeric time axis** — days elapsed since the first observation. This is necessary because statistical models (splines, curve-fitting) work on numbers, not datetime objects.

Step 2 — Explore Seasonal Patterns

Looking at the average price by calendar month reveals a clear annual cycle:

- **Winter (Dec–Mar): ~$11.70–$11.78** — heating demand drives prices up
- **Spring/Summer (Apr–Aug): ~$10.70–$10.90** — demand falls, so prices drop
- **Autumn (Sep–Oct): ~$10.75–$11.08** — prices begin rising again

There's also a visible **upward trend** year-over-year (~$0.54/year) as the market drifted higher across the window.

Step 3 — Fit a Trend + Fourier Seasonal Model

We fit the following model to the 48 monthly observations:

$$\text{price}(t) = a + b \cdot t + c_1 \cdot \sin(2\pi t/365.25) + d_1 \cdot \cos(2\pi t/365.25) \quad \leftarrow \text{1st harmonic} + c_2 \cdot \sin(4\pi t/365.25) + d_2 \cdot \cos(4\pi t/365.25) \quad \leftarrow \text{2nd harmonic}$$

- **Linear trend** ($a + b \cdot t$): captures the multi-year drift. The fitted slope of **+0.00149 USD/day ≈ +$0.54/year** quantifies the upward drift.
- **Fourier harmonics**: represent the annual seasonal cycle as a smooth sinusoidal wave. Two harmonics give the model enough flexibility to capture the asymmetric winter/summer shape without overfitting.
- **Why curve_fit?**: Scipy's curve_fit uses least-squares minimisation (Levenberg-Marquardt) to find optimal parameter values. The result: **RMSE = $0.19, R^2 = 0.94** — a strong fit on just 48 observations.

Step 4 — Cubic Spline for Historical Interpolation

For any date **within** the historical data range, we use a **cubic spline** fitted through all 48 exact month-end prices. A cubic spline:

- Passes exactly through every observed data point
- Is smooth (continuous 1st and 2nd derivatives) between points
- Is far more accurate than the parametric model for "what was the price on 15 June 2022?" type queries

---

Step 5 — The estimate_price() Function

The two models are combined with a simple routing rule:

Date range	Method	Rationale
≤ last observed (Sep 2024)	Cubic spline	Precise interpolation through known prices
Last obs + up to 1 year	Seasonal model	Principled extrapolation using trend + seasonality
Beyond 1 year	Raises ValueError	Too uncertain to be useful

python

price = estimate_price("2025-06-15") # → $12.15 (extrapolated)

price = estimate_price("2022-06-15") # → $10.55 (interpolated)

Step 6 — Visualisation (4 panels)

- **Panel A**: Raw data, spline curve, and the 1-year extrapolation — the dotted vertical line marks the handoff between methods
- **Panel B**: Residuals of the seasonal model — small, patternless bars confirm no systematic bias
- **Panel C**: Bar chart of average price by month — immediately shows the winter premium and summer trough
- **Panel D**: Model decomposition — separates the long-run trend from the oscillating seasonal component, making each driver transparent